Advanced Data Management: what to state, compute, read and argue

Organised like the paper: A quick checks, B situations, C a real plan, D tickets. green = procedure, red ! = trap, [FS] = the formula is on the exam's formula sheet. B pages, N rows, r rows per page, M buffer pages (`work_mem`/8 KB), s selectivity, $k = sN$ matching rows, h tree height, W working set, h_{it} hit ratio.

A Quick checks: the fact behind the lette

Who rejects what: grammar: parser ▹ unknown column, type, permission: binder (catalog) ▹ scan or join choice: planner ▹ wrong estimate: statistics (`ANALYZE`) ▹ low hit ratio: buffer pool ▹ deadlock: transaction manager. The planner never runs the query; the executor never chooses.

Pages: record = columns +24 B, rounded up to 8; $r = \lfloor 8168/(\text{record} + 4) \rfloor$; $B = \lceil N/r \rceil$. VACUUM frees space inside pages, the file shrinks only with VACUUM FULL or CLUSTER.

Index facts: lookup = h index pages +1 heap page; h is small (3–4), never $\log_2 N$. An index serves its key expression only (`lower(email)` needs an index on `lower(email)`); a composite (a, b) serves a leading prefix, not b alone; BRIN helps only a physically ordered column; hash helps only =.

Blocking nodes (all input before the first output): Sort, Hash build, HashAggregate, Materialize. **Streaming**: scans, Filter, Limit, Nested Loop, Merge Join, hash probe, GroupAggregate on sorted input. A LIMIT above a breaker saves nothing below it.

Sort: runs vs fan-in $M - 1$ decides the passes [FS]; doubling M removes at most one pass.

Estimate: equality on an MCV value = its frequency; AND multiplies (independence); never add.

Clock-sweep: `usage_count` 0–5, +1 per hit; the hand decrements and evicts at 0; pinned frames are skipped. Pin = in use now; dirty = must be written first.

! Letter without the one-line reason scores one point of three.

B Situations: compute, then decide

Write: **data → formula → numbers with units → the sentence that quotes the number**. Everything marked [FS] is on the formula sheet; below is what is not.

Index or scan, with the correlation

s, k from the predicate [FS] ▹ read `pg_stats.correlation` ▹ heap pages the matches touch:

scattered (corr ≈ 0) : $B\,(1 - (1 - s)^r)$
compute $(1 - s)^r = e^{r \ln(1-s)}$; $sr \geq 3 \Rightarrow$ nearly every page

contiguous (corr ≈ ±1) : $\lceil k/r \rceil \approx sB$
append-only tables, after CLUSTER

▹ cost the index path [FS] with those pages as its heap term, random price if scattered, sequential if contiguous ▹ compare with the Seq Scan [FS] ▹ between the plain-index threshold and a few percent, a **bitmap scan** reads the same pages once, in order ▹ add the write price: one random page per index per INSERT ▹ decide, quoting both costs. **The one word**: correlation (pages, not rows).

B⁺ tree and latency

reads per lookup = $h + 1$, per range [FS] ▹ time = reads × device (RAM 0.1 μs, SSD 100 μs, HDD 10 ms) ▹ cached levels cost the RAM price ▹ say which target is met and what to buy (SSD, RAM for the index) rather than the pool size.

Sort and "what changes with M"

recompute runs and passes [FS] for each M ▹ one pass needs the input in memory; otherwise at most one pass fewer per doubling ▹ PostgreSQL: `Sort Method` line, `external merge Disk: x` needs `work_mem` ≥ x, set with `SET LOCAL` in the transaction, never globally.

Join comparison

data B_R, B_S, $|R|$, $|S|$, M, d, indexes, existing order ▹ per algorithm settle the fits case from M first, then its formula [FS] ▹ a sort-merge pays the *full* external sort of each unsorted input ▹ minimum, then the trap answers: rows where pages belong (simple NL), M for $M - 2$ ▹ say when the loser wins: more memory (both read each input once), an index on the inner key with a tiny outer, sorted output wanted ▹ for ORDER BY, aggregate first and sort the groups, not the big input. Each join's disaster: block NL when nothing fits; index NL with a big outer (`loops=` huge); sort-merge with nothing sorted and little memory; hash on <, a build $\gg M$ (`Batches` > 1) or a skewed key.

Buffer pool numbers

EAT = $h_{\text{it}}\,T_{\text{hit}} + (1 - h_{\text{it}})\,T_{\text{miss}}$ then "× RAM" and "× the other scenario"; the miss term dominates

h_{it} = hit/(hit + read) `read` is not disk: the OS cache serves many

ring iff $B >$ shared_buffers/4 frames = `shared_buffers`/8 KB; ring = 32 frames; W survives iff $W \leq$ frames − 32

reload after flooding = $W \times T_{\text{miss}}$ under LRU/clock/FIFO a scan > pool evicts all of W; the scan gains nothing (use-once)

Sizing: `shared_buffers` ≈ 25 % of RAM; size for W, not the database; more starves the OS cache and `work_mem`; measure W (`pg_buffercache`) and h_{it} (`pg_statio_user_tables`) first.

Replacement trace

draw the frames ▹ per request: hit (LRU: move to front; FIFO: nothing; clock: set bit) or miss (victim by the rule; dirty victim = one extra write) ▹ count hits, misses, writes ▹ EAT.

! Rows where pages belong. Fan-in M (it is $M-1$), block NL with M (it is $M-2$). Reads only (every sort pass reads and writes). Forgetting pass 0. "Index wins because s is small" without pages. "99 % is a good hit ratio" (compute the EAT). Tracing LRU as FIFO. Doubling memory halves the time.

C A real plan: read, locate, fix

Node line: `(cost=start..total rows=est width)` is the planner's belief; `(actual time=first..last rows=act loops=n)` is measured *per loop*: total rows = act × loops, time = last × loops. `start` = work before the first row (a breaker's whole input).

Sub-lines: `Filter` + `Rows Removed by Filter`: tested after reading the row ▹ `Index Cond`: answered inside the index ▹ `Recheck Cond` + `Heap Blocks: exact=`: bitmap scan, pages touched ▹ `Heap Fetches`: index-only, 0 is ideal ▹ `Hash Cond`/`Merge Cond`/`Join Filter` ▹ `Buckets Batches Memory`: `Batches` > 1 spilled ▹ `Sort Key`, `Sort Method`, `Memory`/`Disk` ▹ `Group Key` ▹ `Buffers: shared hit/read, temp read/written` per node, children included.

Tree: indentation = children; first child = outer or probe, second = inner or build (under `Hash`); execution starts at the leftmost leaf of the deepest breaker.

Reconstruct the query: Scan on `t a` = FROM; `Hash`/`Merge Cond` = ON; `Filter`/`Index Cond` = WHERE; `Group Key` = GROUP BY; `Sort Key` = ORDER BY; `Limit` = LIMIT.

Five moves

1 leaves: scan type, filter, est vs act ▹ 2 up: join algorithm, order, aggregate, sort ▹ 3 the *lowest* node with a large q-error is the cause; everything above inherits it ▹ 4 resources: `Buffers`, `Sort Method`, `Batches`, `loops` of the inner side, `Heap Blocks` ▹ 5 decide: wrong plan from a wrong estimate (statistics fix) / right plan but slow (memory, index, cold cache, CPU-bound) / nothing to fix; name the command.

Where an estimate comes from

scan `rows=` = MCV frequency (or $1/d$, histogram) × N ▹ a `Filter` multiplies by its own selectivity (independence) ▹ join `rows=` [FS] ▹ q-error = max(est/act, act/est); ≥ 10 is the cause; an *under*-estimate buys per-row plans (nested loop, index lookups) for a big input

▹ name the failure and the command:

`stale: n_live_tup <> reltuples, old last_analyze -> ANALYZE t;`

`correlated columns (a determines b; ANALYZE does not help) -> CREATE STATISTICS s (dependencies) ON a, b FROM t; ANALYZE t;`

`skew (rare value estimated at 1/d) -> ALTER TABLE t ALTER COLUMN c SET STATISTICS 1000; ANALYZE t;`

Where the time goes

`Heap Blocks` = the scattered-pages formula on B ▹ inner side: pages per loop ($h+1$)× `loops` = `hit` + `read` ▹ seconds ≈ `read` × the random price (50–100 μs) ▹ compare the node's `actual time` spans to see which node owns the seconds ▹ the overall hit ratio hides a hot node: read `Buffers` per node.

What each fix changes

extended statistics: estimate right → the join and its order change (hash join, build on the small filtered side) ▹ **ANALYZE**: only stale counts ▹ **composite index**: `Filter` moves into `Index Cond`; `Heap Blocks`, estimate and time unchanged if the filter removed nothing ▹ **CLUSTER**: correlation → 1, `Heap Blocks` → $\lceil k/r \rceil$, scan time collapses, the join cost stays; one-off rewrite under lock, decays with new rows ▹ **covering index** (INCLUDE): `Heap Fetches: 0`, no heap pages ▹ **partial index**: tiny, only when the query repeats the predicate ▹ **work_mem**: `Disk/Batches` disappear ▹ **hash join alternative**: build on the smaller *filtered* input (fits in `work_mem`: `Batches: 1`), probe with a sequential scan: sequential pages instead of `loops` random ones.

! Fixing the scan when the join owns the time. "`read` means disk". Inner `rows` read as a total. Blaming the top node. Adding an index for a 10× estimate error. Calling a bitmap heap scan blocking (it streams) or a bitmap index scan streaming (it does not).

D Tickets: claim, mechanism, number

Half a page: **claim** (right, wrong, or both) ▹ **mechanism** (which subsystem, which rule) ▹ **one number** with units ▹ **consequence** and what to do instead ▹ the tempting wrong answer. The arguments the course expects:

"Column store, analytics are slow": a scan of c of k columns reads c/k of the bytes, compressed further; an insert writes k places instead of 1, a point read reassembles from k. Both workloads exist: keep the row store for the app, give analytics a column copy (COPY to Parquet + DuckDB) or, inside PostgreSQL, BRIN on the time column and a covering index.

"LRU is optimal, replace clock-sweep": OPT (evict the furthest next use) is optimal and needs the future; LRU approximates it with the past; exact LRU costs a list move under a lock on *every* hit, which at 99 % hits is the bottleneck; clock sets a bit, clock-sweep adds frequency via `usage_count`; all of them fail on a scan larger than the pool, which PostgreSQL fixes by intent (rings), not by policy.

"Freeze the join order" (`join_collapse_limit = 1`): n tables have $n!$ left-deep orders (9! = 362,880) but System-R DP costs $2^n - 1$ subsets (511) plus interesting orders; the written order ignores selection pushdown (the most filtered input should come first) and goes stale with the data; fix the one query with `SET LOCAL`, a prepared statement or a materialised intermediate, never server-wide.

"Doubling work_mem halves the sort": passes are a ceiling of a logarithm; a doubling removes at most one merge pass; only $M \geq B$ makes it one pass; find the `Disk` figure and give exactly that.

"Hash joins are buggy, switched off": `enable_*` is a 10^{10} cost penalty applied to every query; with memory both algorithms read each input once and tie; a correctness bug is reproduced and filed, not planned around.

"An index on a small filter never helps / always helps": pages, not rows: scattered matches touch $B(1-(1-s)^r)$ pages at the random price, contiguous ones sB; read the correlation; a bitmap scan sits between; every index costs a random write per insert; drop `idx_scan = 0`.

"More shared_buffers": a use-once scan has hit ratio 0 at any size and is ringed anyway; the pool should hold W, which is measured, not guessed; more pool starves the OS cache (which serves `reads`) and makes checkpoints bursty.

"The slow morning / the nightly scan": flooding under LRU reloads W at T_{miss} each; with rings W survives and the cause is elsewhere (OS cache, a COPY ring, a checkpoint): measure with `pg_buffercache` after the job.

"Statistics are fresh, so the estimate is right": fresh single-column frequencies still multiply under independence; correlated columns need `CREATE STATISTICS`; a wrong leaf poisons every node above it.

"The second run is fast, so the query is fine": the cache, not the plan; compare `Buffers` hit and read; CPU-bound queries (all hits) need a better plan or index, not more memory.

"VACUUM will shrink the table": it frees space inside pages and truncates trailing empty pages only; VACUUM FULL or CLUSTER rewrite; CLUSTER also sets correlation to 1.

"Commit wrote the data": steal + no-force + WAL: commit forces the log, not the pages; committed data is safe in the log; redo the committed, undo the rest.

! A ticket answered without a number. Agreeing or refusing without the mechanism. Not saying what to do instead.

Commands to name

```
EXPLAIN (ANALYZE, BUFFERS) q; -- DML: BEGIN; ... ROLLBACK;
SELECT attname, n_distinct, most_common_vals, most_common_freqs, histogram_bounds, correlation FROM pg_stats WHERE tablename = 't';
SELECT reltuples, relpages FROM pg_class WHERE relname = 't'; SELECT pg_relation_size('t') / 8192;
ANALYZE t; CREATE STATISTICS s (dependencies) ON a, b FROM t; ANALYZE t; ALTER TABLE t ALTER COLUMN c SET STATISTICS 1000;
BEGIN; SET LOCAL work_mem = '256MB'; ...; COMMIT; SET enable_hashjoin = off; RESET ALL;
CREATE INDEX ON t (a, b); ON t (lower(email)); ON t (a) WHERE status = 'open'; ON t (a) INCLUDE (b); ON t USING brin (ts); CLUSTER t USING idx; DROP INDEX i;
SELECT indexrelname, idx_scan FROM pg_stat_user_indexes WHERE relname = 't'; SELECT heap_blks_hit, heap_blks_read FROM pg_statio_user_tables WHERE relname = 't';
CREATE EXTENSION pg_buffercache; SELECT c.relname, count(*) FROM pg_buffercache b JOIN pg_class c ON b.relfilenode = pg_relation_filenode(c.oid) GROUP BY 1 ORDER BY 2 DESC;
```